

PROJECT REPORT

Wireshark – Packet Sniffer and Network Protocol Analyzer

Network Traffic Capture, Protocol Analysis & Packet-Level Security Investigation

Project ID: PTID-CSPP-JUL-26-486

Submitted By

1. Manohar Chowdary
2. Vijay Yesupogu
3. Indu Priya Lalam

Department of Cyber Security-Skillogic
Batch ID: 23-MAR-26-HYD-OFF (Cyber Security) 2026

Acknowledgement

We express our sincere gratitude to Skillogic for providing us with the opportunity to undertake this project titled “Wireshark – Packet Sniffer and Network Protocol Analyzer: Network Traffic Capture, Protocol Analysis & Packet-Level Security Investigation” as part of our Cyber Security training program. This project has provided us with valuable practical exposure to packet capture, protocol analysis, and network traffic inspection using industry-standard tools.

We would like to express our heartfelt gratitude to our trainer and project guide, Mr. Surya Sadanala, for his invaluable guidance, constant encouragement, and technical support throughout the development of this project. His expertise, constructive suggestions, and mentorship played a significant role in helping us understand the concepts and successfully complete this work.

We also extend our sincere thanks to the entire Skillogic team for providing excellent training resources, a practical learning environment, and continuous support throughout the course.

We are grateful to our institution, family members, and friends for their encouragement, motivation, and support during the completion of this project.

Finally, we acknowledge the dedication, teamwork, and collective efforts of all the project members, whose cooperation made the successful completion of this project possible.

Team Members:

1. Manohar Chowdary
2. Vijay Yesupogu
3. Indu Priya Lalam

Abstract

Network traffic carries a continuous stream of information about how systems communicate, what services are exposed, and how data moves across an infrastructure. Without the ability to observe this traffic at the packet level, security teams and administrators are left blind to a large portion of what is actually happening on their networks. This project focuses on Wireshark, the world's most widely used open-source network protocol analyzer, which enables the capture, decoding, and inspection of network traffic in real time or from previously recorded capture files.

The primary objective of this project is to study the architecture, features, installation process, and practical applications of Wireshark in a controlled and authorized environment. The project demonstrates core packet-analysis workflows, including live packet capture, the use of capture and display filters, protocol hierarchy statistics, TCP stream reconstruction, and the identification of plaintext credentials, DNS activity, and suspicious ARP traffic. Each technique is performed using the Wireshark graphical interface, and the resulting captures are analysed to understand the security posture of the traffic being observed.

A virtual laboratory environment was established using authorized test machines to ensure that all packet captures were conducted safely and ethically. The observations obtained from each capture provide valuable insight into the protocols in use, the presence of unencrypted or sensitive data, and behaviours that could indicate misconfiguration or malicious activity. Based on these findings, appropriate security recommendations are discussed to improve the overall security posture of the target environment.

This project highlights the significance of Wireshark as an efficient and versatile tool for network troubleshooting, protocol analysis, digital forensics, and security auditing. It demonstrates how deep packet inspection can assist organizations in verifying encryption practices, detecting credential leakage, understanding application behaviour, and supporting incident response. The knowledge and practical experience gained through this project provide a strong foundation for understanding network traffic analysis and its role in protecting modern information systems.

Table of Contents

Table of Contents

Chapter 1 — Introduction
Chapter 2 — Problem Statement
Chapter 3 — Objectives
Chapter 4 — Nmap Overview
Chapter 5 — Installation
Chapter 6 — Methodology
Chapter 7 — Practical Demonstration
Chapter 8 — Risk Analysis
Chapter 9 — Conclusion

List of Figures

Figure No.	Figure Title
Figure 4.1 & 4.3	➤ Wireshark Architecture and Packet Processing Flow. ➤ Wireshark main interface showing a live capture.
Figure 5.1	➤ Wireshark main interface showing a live capture. ➤ Verification of Wireshark Installation and Non-Root Capture Setup
Figure 5.2	➤ Wireshark Installation on Microsoft Windows with Npcap
Figure 6.2	➤ Project Methodology Flowchart. ➤ Test Environment Network Topology
Figure 7.1	➤ Wireshark Interface.
Figure 7.2	➤ Live Packet Capture on the Selected Interface.
Figure 7.3	➤ Ping to generate ICMP on Target machine. ➤ ICMP packet Inspection.
Figure 7.4	➤ ARP Traffic Inspection and Spoofing Indicators
Figure 7.5	➤ ARP Traffic Inspection and Spoofing Indicators. ➤ TCP Conversation statistics.
Figure 7.6	➤ DNS resolution on google. ➤ Metasploitable web server. ➤ Live http packet capture. ➤ DNS filter applied, showing DNS packets
Figure 7.7	➤ TCP Stream window showing the HTTP conversation
Figure 7.8	➤ Protocol Hierarchy Statistics
Figure 7.9	➤ Reconstructed session Using Tcp Stream.
Figure 7.10	➤ Plaintext Credentials Entered on website ➤ Plaintext Credentials captured over http.
Figure 7.11	➤ DNS Query and Response Analysis.

Chapter 1: Introduction

Understanding what is actually happening on a network is a foundational requirement of cyber security. While tools such as firewalls and intrusion detection systems raise alerts, the raw packets travelling across the wire tell the complete story — every request, response, handshake, and byte of data exchanged between hosts. This project focuses on Wireshark, the world's foremost open-source network protocol analyser, used for capturing and inspecting network traffic at a granular level.

Wireshark was originally released as Ethereal in 1998 and has since become the de facto standard for network troubleshooting, protocol development, education, and security analysis. It captures packets using the underlying operating system's packet-capture library (libpcap on Linux/macOS, Npcap on Windows) and decodes them using a large library of protocol dissectors, presenting the data in a structured, human-readable format.

This project explores Wireshark from both a conceptual and practical standpoint — covering its role in cyber security, how it is installed and configured, the methodology behind capturing and filtering traffic, and a hands-on demonstration of its core capabilities against a test environment.

Chapter 2: Problem Statement

Organizations today face significant challenges in gaining visibility into what is actually transmitted across their networks. Some of the key problems that motivate this project include:

- **Lack of visibility into raw network traffic** — administrators often rely on summarized logs rather than the underlying packets, making it difficult to verify exactly what data is being transmitted and to whom.
- **Undetected transmission of sensitive data in plaintext** — protocols such as HTTP, FTP, and Telnet can expose usernames, passwords, and other sensitive information if left unencrypted, often without anyone noticing.
- **Difficulty identifying abnormal or malicious traffic patterns** — activity such as ARP spoofing, DNS tunnelling, or unusual outbound connections can go unnoticed without packet-level inspection.
- **Challenges in network troubleshooting** — application and connectivity issues are often difficult to diagnose without observing the actual handshake and protocol exchange between hosts.
- **Absence of a low-cost, flexible tool for packet-level auditing** — especially for smaller teams, students, and organizations with limited budgets for commercial network forensics platforms.

These challenges highlight the need for a reliable, well-understood tool that can be used to capture, decode, and analyse network traffic in detail — which is precisely the gap that Wireshark is designed to fill.

Chapter 3: Objectives

The objectives of this project are as follows:

- To understand the architecture, features, and working principles of Wireshark as a network protocol analyzer.
- To install and configure Wireshark in a controlled lab/test environment.
- To study the methodology behind capturing traffic and applying capture and display filters.
- To perform practical packet captures on a test network and document the actions, outputs, and observations.
- To analyse how Wireshark contributes to identifying sensitive data exposure, protocol misuse, and suspicious traffic behaviour.
- To present findings in a structured, professional report format suitable for academic evaluation.

Chapter 4: Wireshark Overview

4.1 What is Wireshark?

Wireshark is a free and open-source network protocol analyzer used to capture and interactively browse traffic running on a computer network. It is widely used by network administrators, security professionals, and developers for troubleshooting connectivity issues, analysing protocol behaviour, and investigating security incidents.

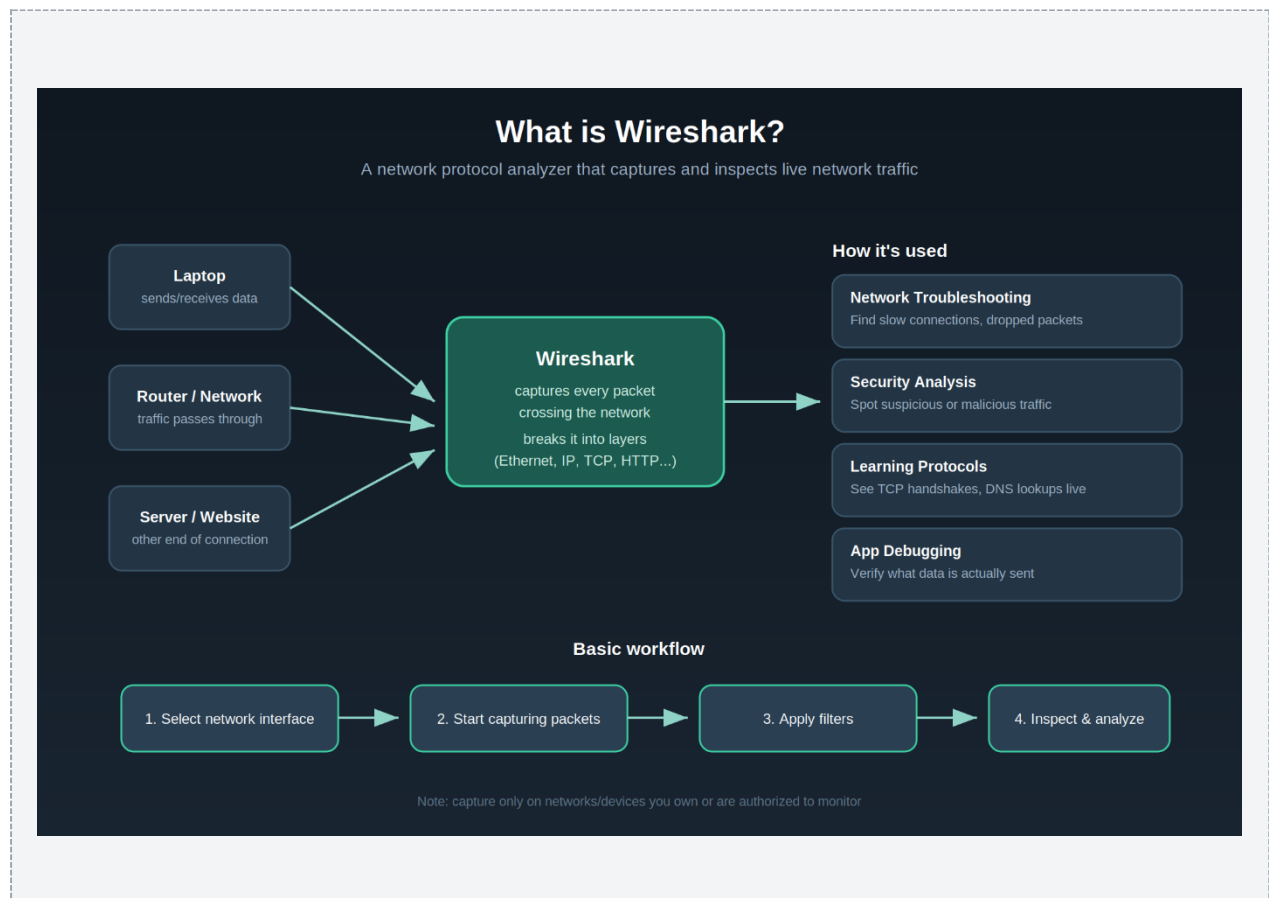


Figure 4.1 Wireshark Architecture and Packet Processing Flow

4.2 Key Features

- **Live Packet Capture:** Captures traffic in real time from wired, wireless, or virtual network interfaces.
- **Deep Protocol Dissection:** Decodes hundreds of protocols, from Ethernet and IP up to application-layer protocols such as HTTP, DNS, and TLS.

- **Follow Stream:** Reconstructs entire TCP, UDP, HTTP, or TLS conversations into a readable, linear view.
- **Statistics Tools:** Provides protocol hierarchy views, conversation and endpoint statistics, and I/O graphs for traffic analysis.
- **Colour-Coding Rules:** Highlights packets based on protocol or condition, making anomalies easier to spot visually.
- **Import/Export Support:** Reads and writes standard capture formats such as .pcap and .pcapng for interoperability with other tools.

4.3 Relevance to Cyber Security

Wireshark plays a central role in network forensics, incident response, and security auditing. Analysts use it to verify whether sensitive data is being transmitted securely, to reconstruct the sequence of events during a security incident, to detect abnormal traffic patterns such as spoofing or scanning activity, and to validate that encryption is functioning as intended across the network.

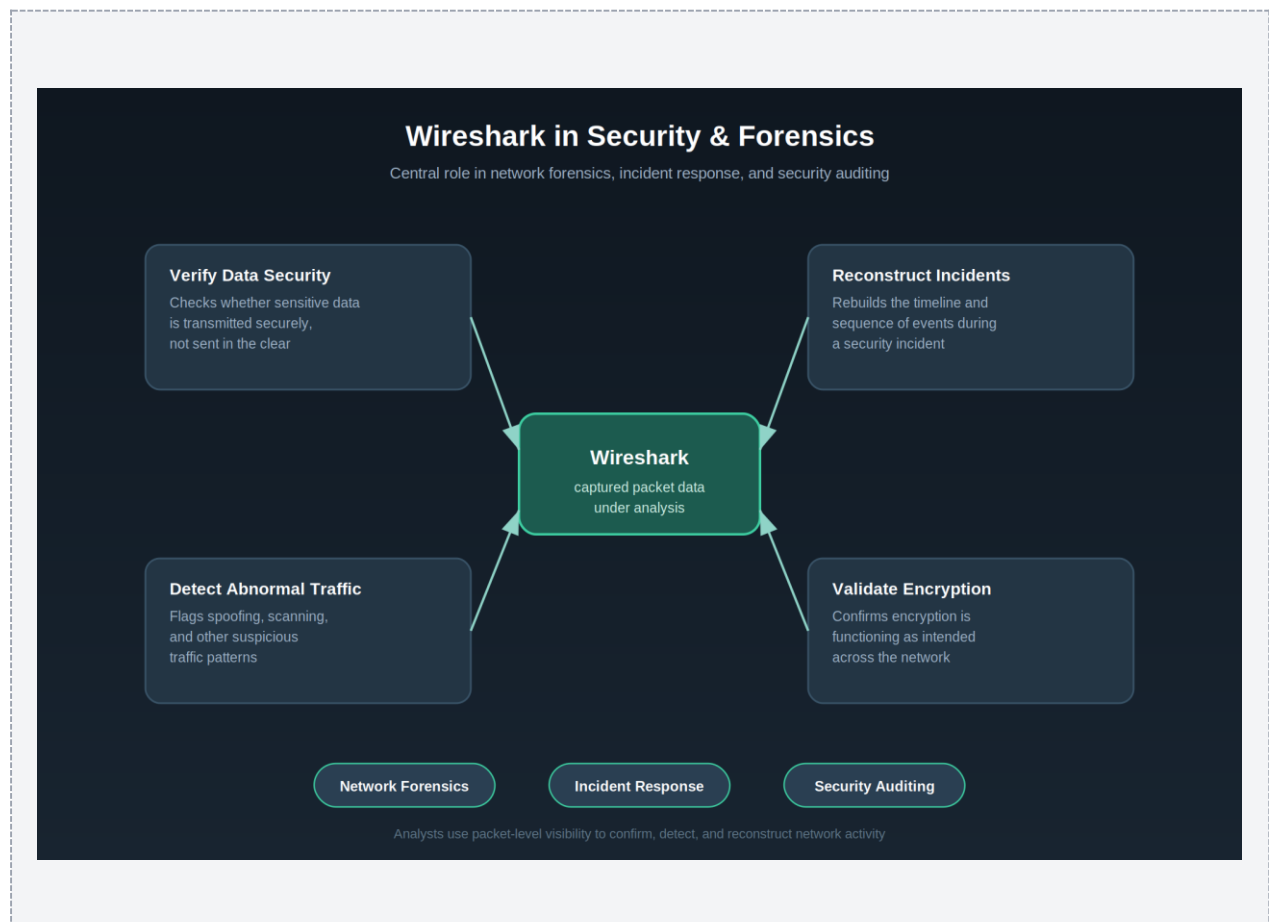


Figure 4.3 Wireshark Main Interface Showing a Live Capture

Chapter 5: Installation

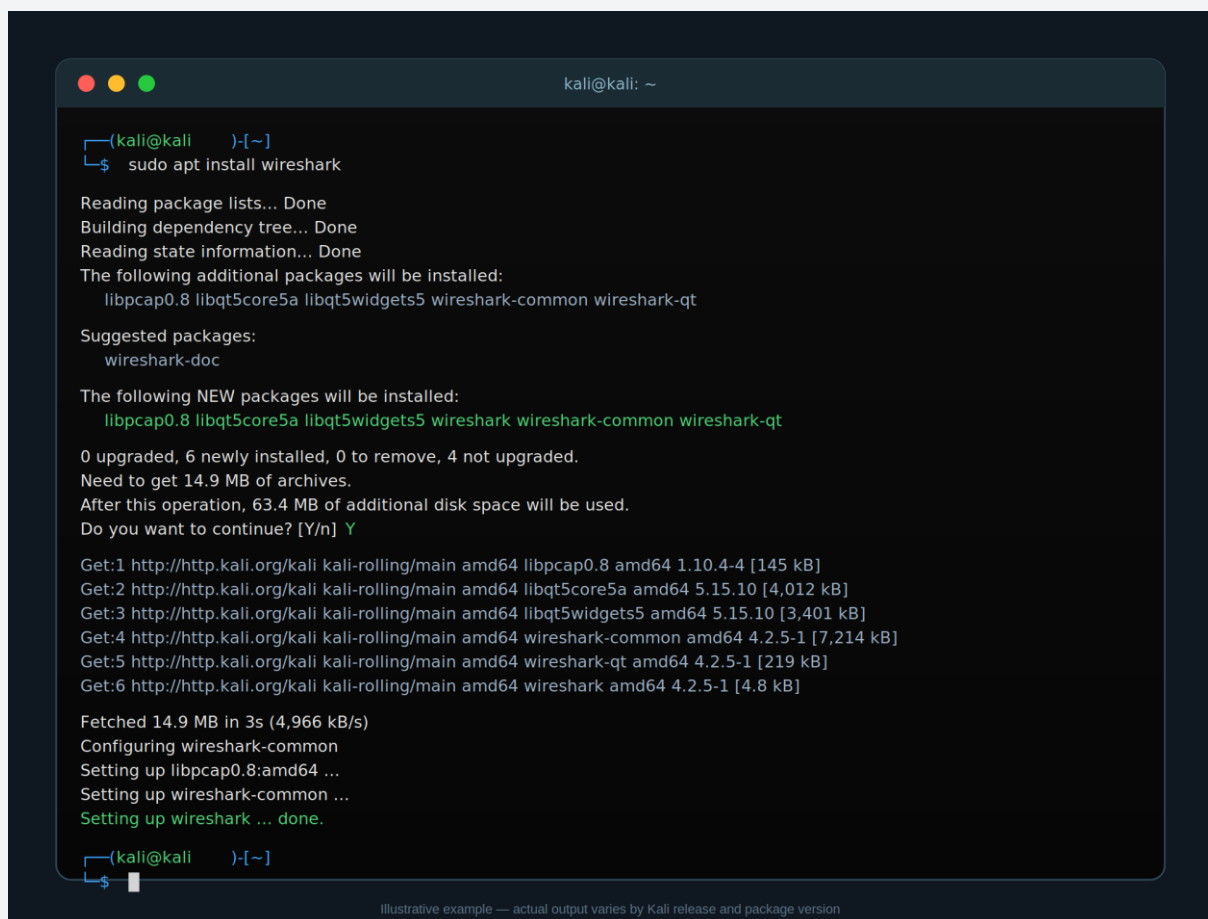
Wireshark is available for Windows, Linux, and macOS. The installation steps used for this project are outlined below.

5.1 Installation on Linux

```
$ sudo apt update
```

```
$ sudo apt install wireshark -y
```

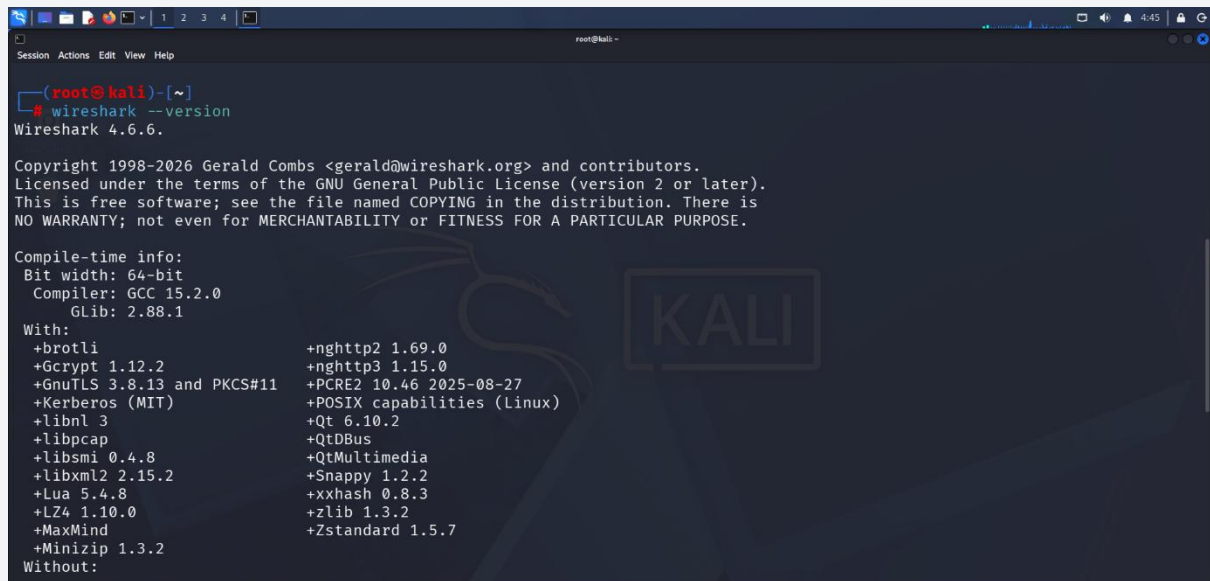
```
$ wireshark --version
```



```
kali@kali: ~  
(kali@kali) ~  
$ sudo apt install wireshark  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
  libpcap0.8 libqt5core5a libqt5widgets5 wireshark-common wireshark-qt  
Suggested packages:  
  wireshark-doc  
The following NEW packages will be installed:  
  libpcap0.8 libqt5core5a libqt5widgets5 wireshark wireshark-common wireshark-qt  
0 upgraded, 6 newly installed, 0 to remove, 4 not upgraded.  
Need to get 14.9 MB of archives.  
After this operation, 63.4 MB of additional disk space will be used.  
Do you want to continue? [Y/n] Y  
Get:1 http://http.kali.org/kali kali-rolling/main amd64 libpcap0.8 amd64 1.10.4-4 [145 kB]  
Get:2 http://http.kali.org/kali kali-rolling/main amd64 libqt5core5a amd64 5.15.10 [4,012 kB]  
Get:3 http://http.kali.org/kali kali-rolling/main amd64 libqt5widgets5 amd64 5.15.10 [3,401 kB]  
Get:4 http://http.kali.org/kali kali-rolling/main amd64 wireshark-common amd64 4.2.5-1 [7,214 kB]  
Get:5 http://http.kali.org/kali kali-rolling/main amd64 wireshark-qt amd64 4.2.5-1 [219 kB]  
Get:6 http://http.kali.org/kali kali-rolling/main amd64 wireshark amd64 4.2.5-1 [4.8 kB]  
Fetched 14.9 MB in 3s (4,966 kB/s)  
Configuring wireshark-common  
Setting up libpcap0.8:amd64 ...  
Setting up wireshark-common ...  
Setting up wireshark ... done.  
(kali@kali) ~  
$
```

Illustrative example — actual output varies by Kali release and package version

Figure 5.1(1) Wireshark Installation on Linux Using APT Package Manager



```

root@kali: ~
# wireshark --version
Wireshark 4.6.6.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Compile-time info:
  Bit width: 64-bit
  Compiler: GCC 15.2.0
  Glib: 2.88.1
  With:
    +brotli
    +Gcrypt 1.12.2
    +GnuTLS 3.8.13 and PKCS#11
    +Kerberos (MIT)
    +libnl 3
    +libpcap
    +libsmi 0.4.8
    +libxml2 2.15.2
    +Lua 5.4.8
    +LZ4 1.10.0
    +MaxMind
    +Minizip 1.3.2
    +nghttp2 1.69.0
    +nghttp3 1.15.0
    +PCRE2 10.46 2025-08-27
    +POSIX capabilities (Linux)
    +Qt 6.10.2
    +QtDBus
    +QtMultimedia
    +Snappy 1.2.2
    +xxhash 0.8.3
    +zlib 1.3.2
    +Zstandard 1.5.7
  Without:

```

Figure 5.1(2) Verification of Wireshark Installation and Non-Root Capture Setup

5.2 Installation on Windows

On Windows, Wireshark is installed using the official installer from [wireshark.org](https://www.wireshark.org), which also bundles the Npcap packet-capture driver required for live capturing.

- Download the Windows installer from the official Wireshark website.
- Run the installer and accept the license agreement.
- Ensure the Npcap driver option is selected during installation, as it is required for live packet capture.
- Complete the installation and verify by opening Wireshark and checking Help > About Wireshark.

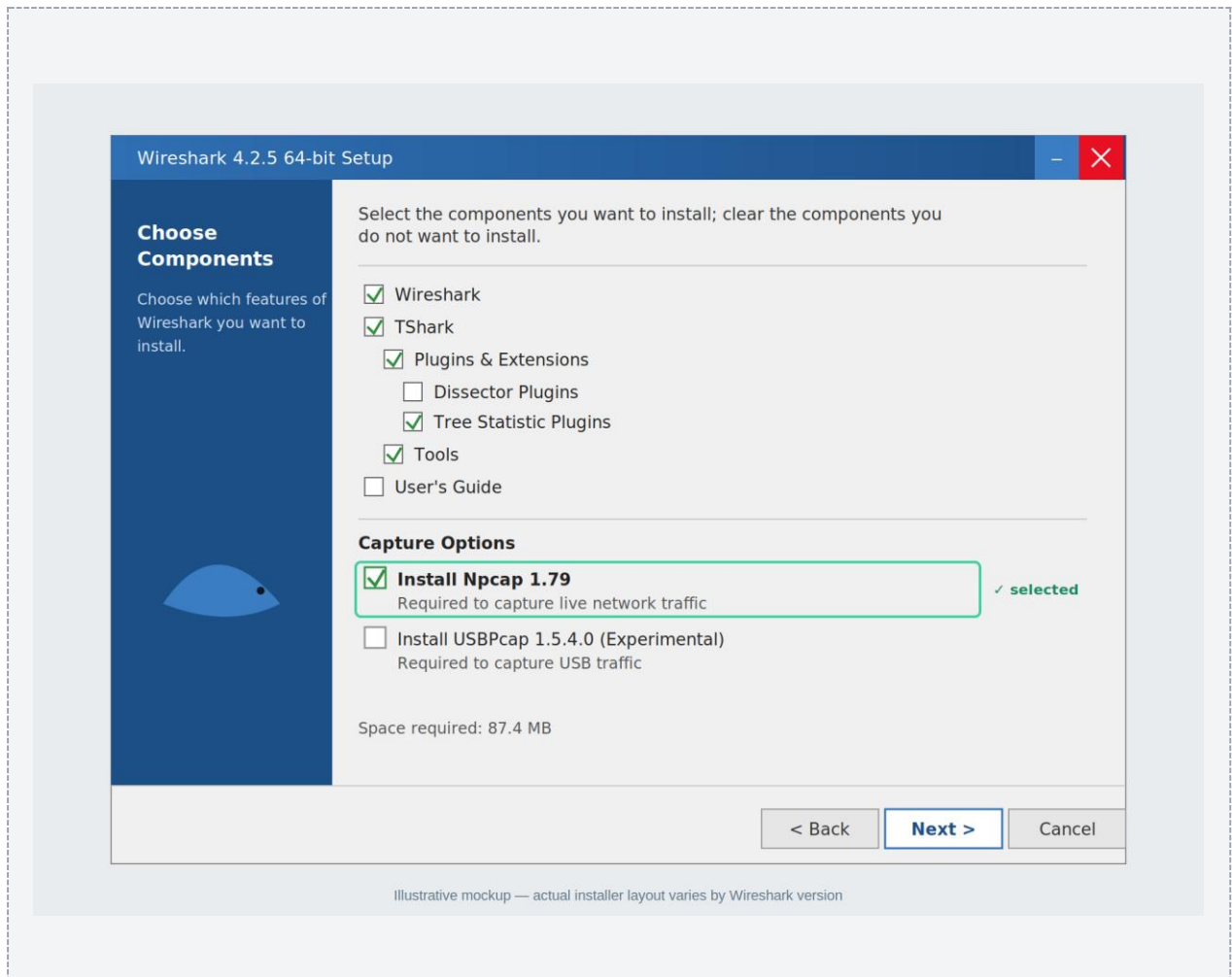


Figure 5.2 Wireshark Installation on Microsoft Windows with Npcap

Chapter 6: Methodology

The project followed a structured methodology to ensure that packet capture activity was conducted safely, ethically, and within a controlled environment.

6.1 Test Environment

All captures in this project were performed only on networks and systems the team owns or is explicitly authorized to monitor (e.g., a local virtual machine lab or a designated test network). Capturing traffic on networks without authorization may violate wiretapping and privacy laws, and was strictly avoided.

6.2 Capture and Analysis Approach

- **Step 1 — Interface Selection:** Identify and select the appropriate network interface on which traffic will be captured.
- **Step 2 — Start Capture:** Begin a live capture session and generate representative traffic within the test environment.
- **Step 3 — Apply Filters:** Use capture and display filters to isolate traffic relevant to the analysis (e.g. a specific protocol, host, or port).
- **Step 4 — Protocol Analysis:** Examine the packet details and hex panes to understand the structure and content of each protocol layer.
- **Step 5 — Stream Reconstruction:** Use Follow Stream to reconstruct full conversations for deeper inspection of application-layer data.
- **Step 6 — Statistics & Reporting:** Use built-in statistics tools to summarise protocol usage and traffic patterns.
- **Step 7 — Documentation:** Record each action performed, capture relevant output, and analyse the results.

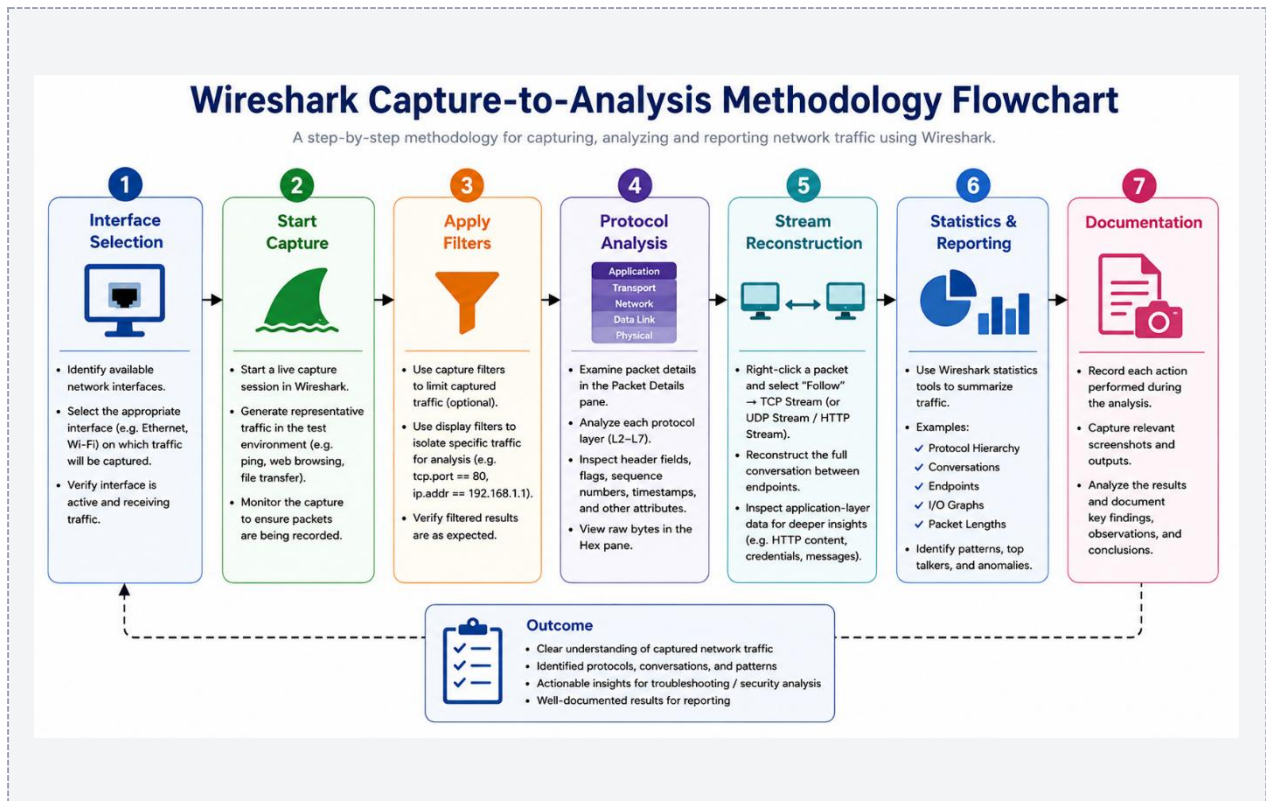


Figure 6.2(1) Project Methodology Flowchart

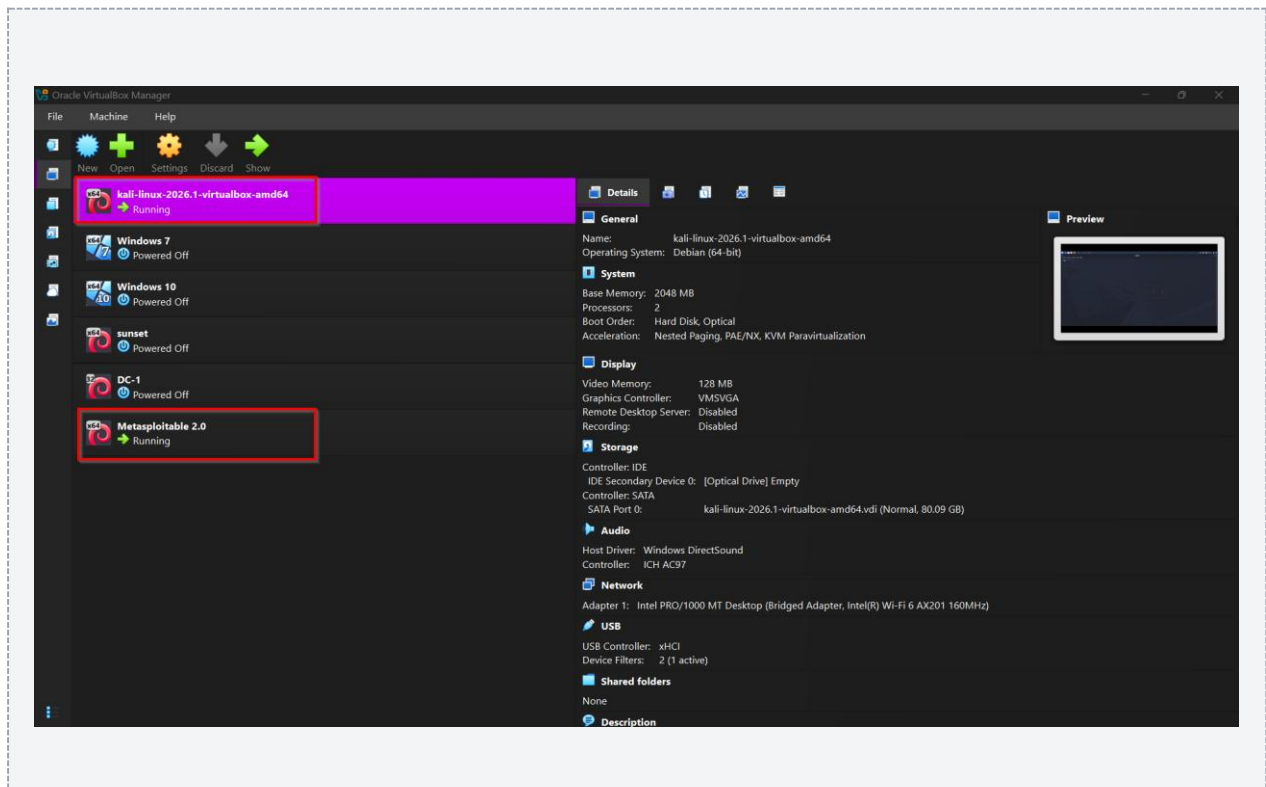


Figure 6.2(2) Test Environment Network Topology

6.3 Tools Used

- Wireshark (GUI) for interactive packet capture and analysis.
- Metasploitable as a Target Machine to send packets and capture them using Wireshark.
- A virtual lab environment (e.g., VirtualBox/VMware) with isolated test machines.
- A text editor/terminal for recording commands, filters, and observations.

Chapter 7: Practical Demonstration

This chapter presents the hands-on packet captures performed as part of this project. For each scenario, the purpose, the action or filter used, a placeholder for the corresponding screenshot, and observations are provided.

7.1 Starting a Wireshark

Purpose: To Start wireshark using kali terminal, so that live traffic on the selected network interface can be observed in the range of protocols present on the network.

```
$ wireshark
```

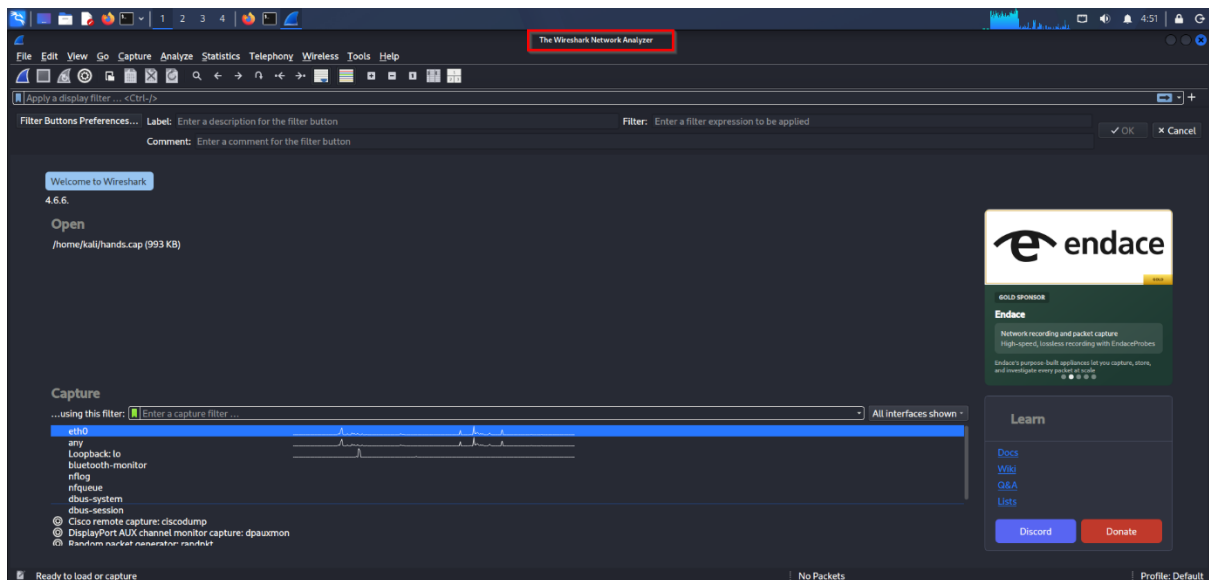


Figure 7.1 Wireshark Interface

7.2 Starting a Wireshark and Capturing Traffic

Purpose: This section documents a hands-on packet capture and analysis exercise carried out in the Kali Linux environment using Wireshark. The exercise generates DNS and HTTP traffic, captures it, and applies Wireshark's filtering, stream-reassembly, and statistical-analysis features to examine the traffic in detail, using a Metasploitable 2 virtual machine as the lab target.

```
$ Capture > Options > select interface (eth0) > Start
```

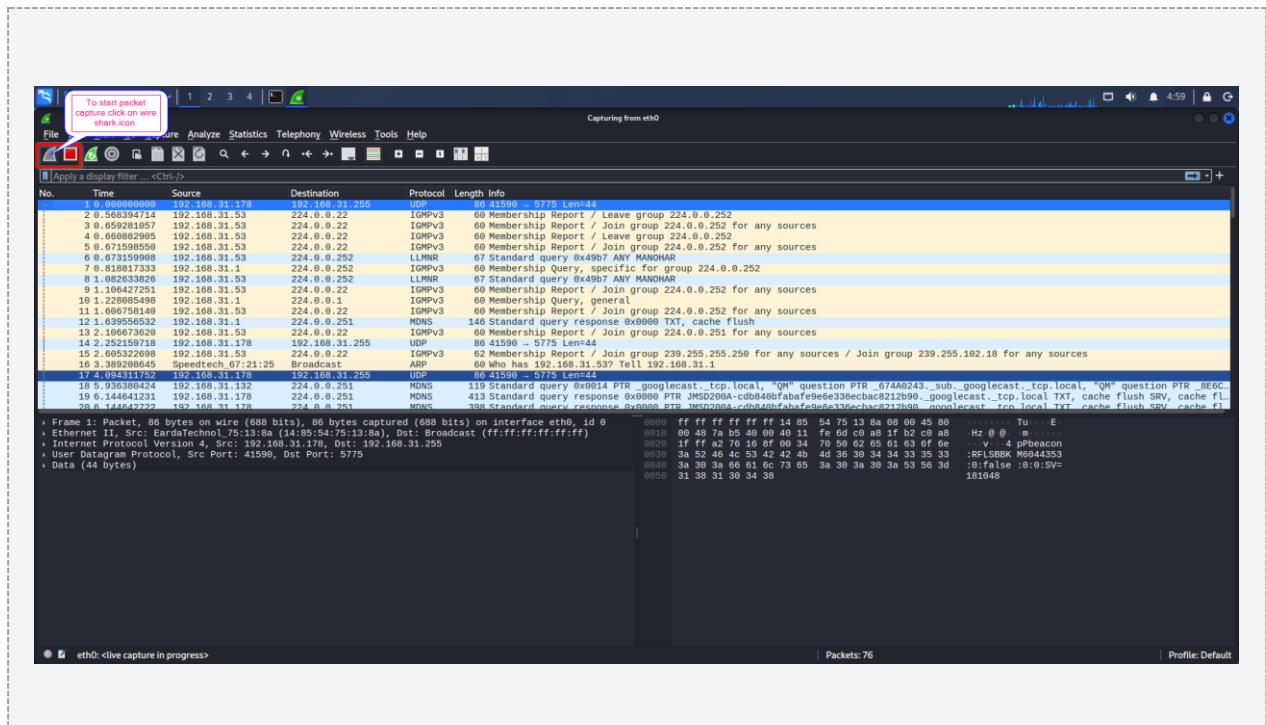


Figure 7.2 Live Packet Capture on the Selected Interface

Observation: The capture successfully collected traffic across multiple protocols, including ARP, TCP, UDP, DNS, and HTTP. Within a short capture window, several hundred packets were recorded, confirming that the interface and capture driver were configured correctly.

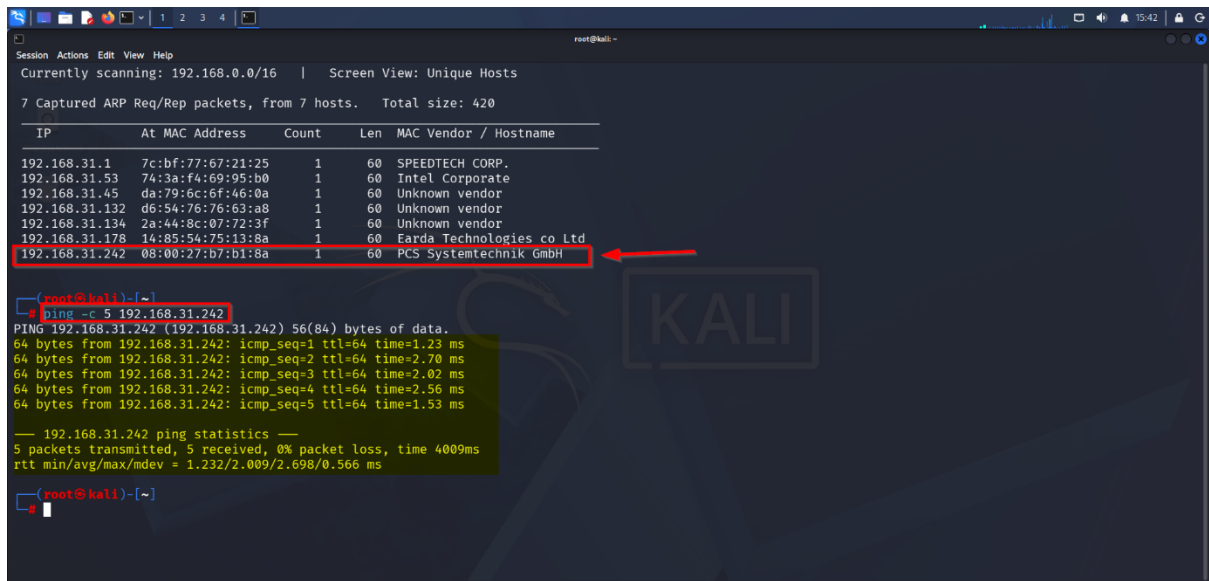
7.3 ICMP Capturing

Purpose: To capture and analyse ICMP (Internet Control Message Protocol) packets exchanged between Kali Linux and the Metasploitable target system. This helps verify network connectivity and understand how ICMP Echo Request and Echo Reply packets are transmitted. Wireshark is used to monitor and inspect the captured ICMP traffic.

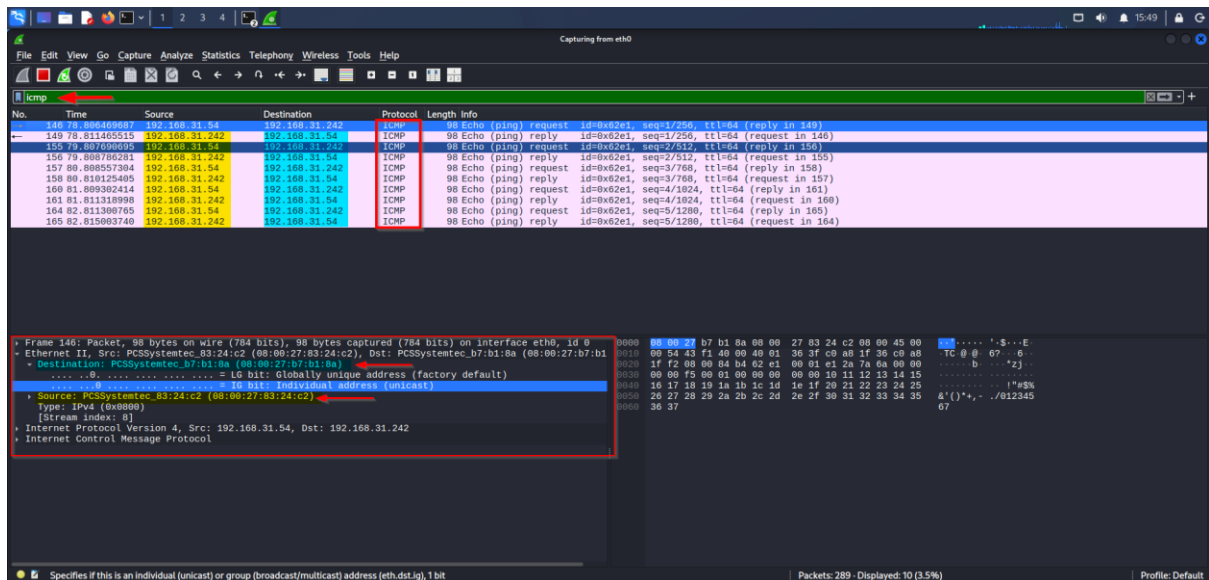
To generate ICMP traffic, we run the following command in **Kali Linux**:

```
$ ping <target_System_IP>
```

Here, 192.168.31.242 is the IP address of the **Metasploitable target system**. While the ping command is running, Wireshark is started on the **eth0 interface** to capture the packets. The display filter **icmp** is then applied to show only ICMP traffic.



7.3(1) Ping to generate ICMP on Target machine



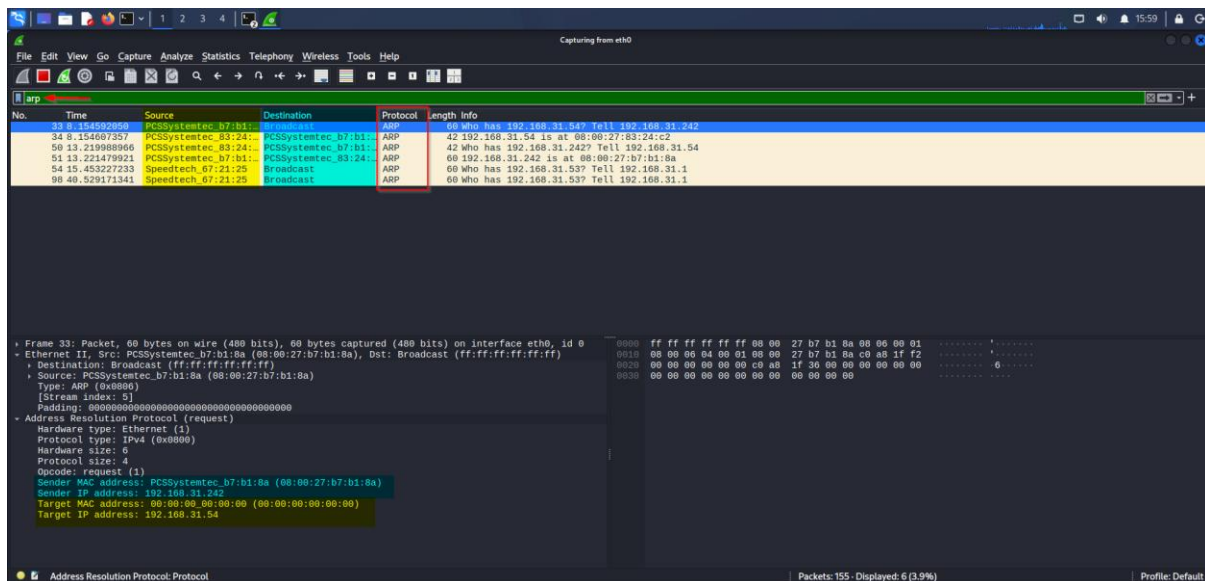
7.3(2) ICMP packet Inspection

Observation: The capture shows **ICMP Echo (ping) requests** from Kali Linux (192.168.31.54) to Metasploitable (192.168.31.242). The target responds with **ICMP Echo Reply** packets, confirming successful network communication. The Wireshark packet details identify the **source and destination IP addresses** and the ICMP protocol. The capture also displays Ethernet-layer source and destination MAC addresses, along with ICMP sequence numbers and TTL values.

7.4 ARP Capturing

Purpose: To capture and analyse ARP (Address Resolution Protocol) traffic between Kali Linux and the Metasploitable target system. ARP is used to resolve an IP address to its corresponding MAC address within the local network. Wireshark helps observe ARP requests and replies and understand the address-resolution process.

To generate and capture ARP traffic, Wireshark is started on the **eth0** interface in Kali Linux, and the display filter: “*arp*” is applied to display only ARP packets.



7.4 ARP Traffic Inspection and Spoofing Indicators

Observation: The capture shows multiple ARP request and reply packets on the local network. The screenshot shows an ARP request asking “Who has 192.168.31.242? Tell 192.168.31.54”, indicating that Kali is resolving the target's MAC address. Wireshark displays the **sender IP/MAC and target IP/MAC** fields; the target MAC is initially unknown in the ARP request. The ARP reply provides the MAC address associated with the requested IP, allowing communication between the systems at the Ethernet layer.

7.5 TCP Packet Analysis

Purpose: To capture and analyse TCP (Transmission Control Protocol) traffic between Kali Linux and the Metasploitable target system. This helps identify TCP connection attempts, responses, and the ports involved in communication. Wireshark is used to examine TCP packets and understand the communication between the source and target.

To capture TCP traffic, Wireshark was started on the **eth0 interface** in Kali Linux, and the display filter: “*tcp*” was applied to display only TCP packets. TCP traffic was then generated between Kali Linux (192.168.31.54) and the Metasploitable target (192.168.31.242). The captured packets were analysed using Wireshark, including the TCP flags, source and destination ports, sequence numbers, and acknowledgements. The **TCP Conversations** statistics were also examined to identify communication between different ports.

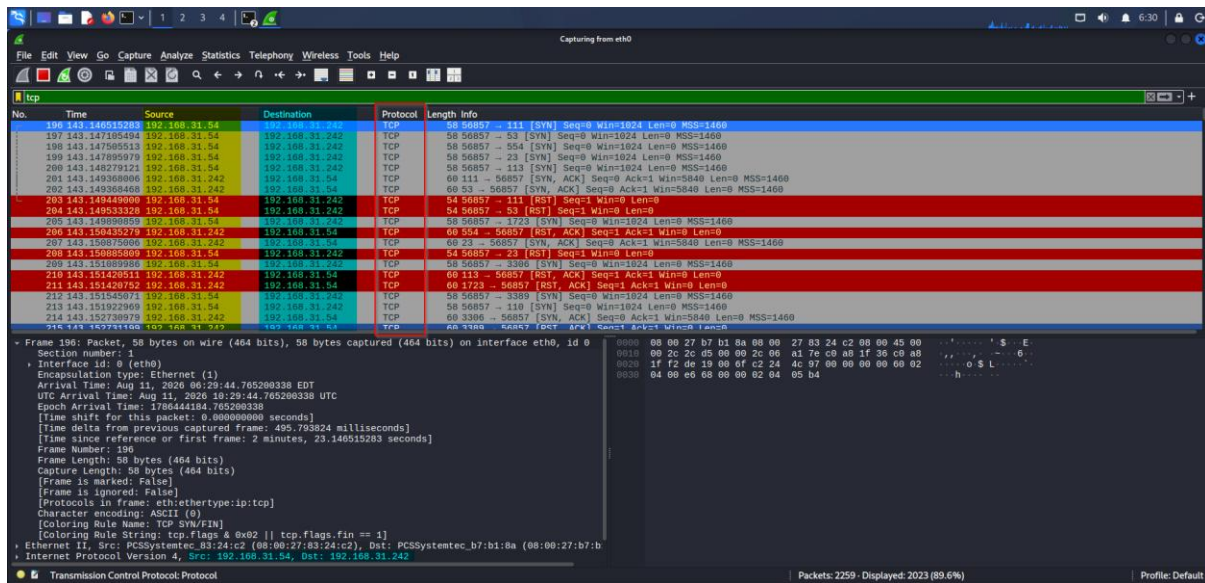


Figure 7.5(1) TCP Traffic between kali and Metasploitable

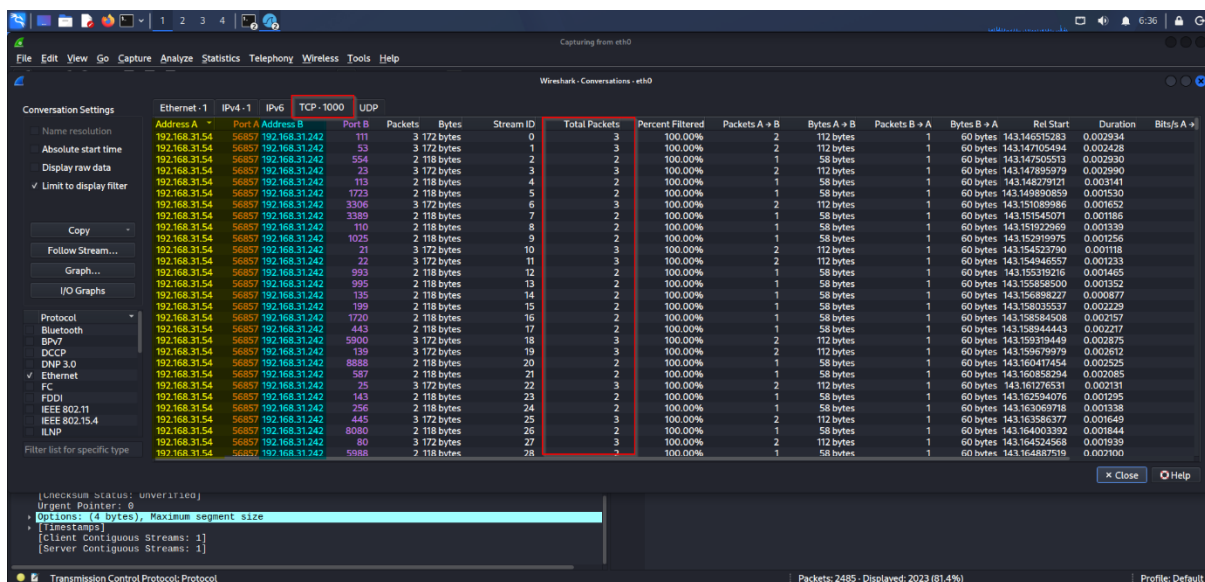


Figure 7.5(2) TCP Conversation statistics

Observation: The capture shows TCP communication between **192.168.31.54** (Kali Linux) and **192.168.31.242** (Metasploitable). Multiple **SYN**, **SYN-ACK**, **ACK**, and **RST** packets are visible, indicating TCP connection attempts and

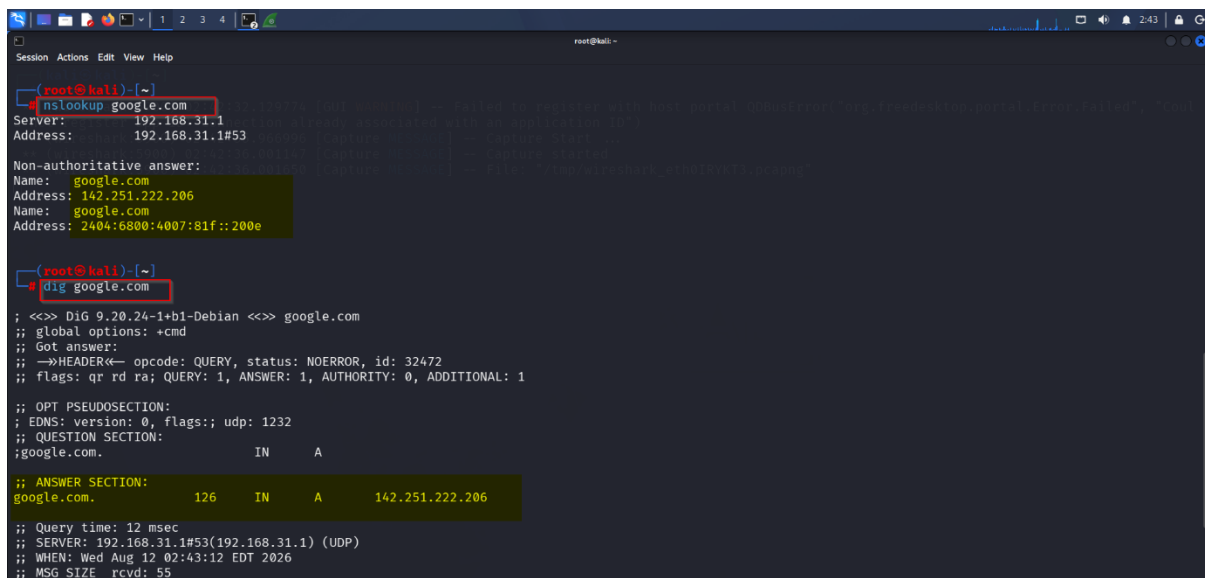
responses from the target. The TCP Conversations window shows communication across several ports, including **21, 22, 23, 25, 80, 110, 111, 139, 443, 3306, 5432, 5900, 8080**, and others. Most conversations contain **2–3 packets**, showing connection attempts and responses, which provides useful information about the TCP services exposed by the Metasploitable system.

7.6 HTTP Packet Analysis

Purpose: To capture and analyse **HTTP traffic** generated while accessing web resources from Kali Linux. This helps understand how HTTP requests and responses are exchanged between a client and a web server. Wireshark is used to inspect HTTP methods, source and destination addresses, requested URIs, and HTTP status codes.

First, DNS resolution was performed in Kali Linux using:

```
$ nslookup google.com
or
$ dig google.com
```



```

root@kali: ~
nslookup google.com
Server: 192.168.31.1
Address: 192.168.31.1#53

Non-authoritative answer:
Name: google.com
Address: 142.251.222.206
Name: google.com
Address: 2404:6800:4007:81f::200e

root@kali: ~
dig google.com

;<<>> DiG 9.20.24-1+b1-Debian <<>> google.com
;; global options: +cmd
;; Got answer:
-->HEADER<-- opcode: QUERY, status: NOERROR, id: 32472
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;google.com. IN A

;; ANSWER SECTION:
google.com. 126 IN A 142.251.222.206

;; Query time: 12 msec
;; SERVER: 192.168.31.1#53(192.168.31.1) (UDP)
;; WHEN: Wed Aug 12 02:43:12 EDT 2026
;; MSG SIZE rcvd: 55

```

Figure 7.6(1) DNS resolution on google

The commands resolved google.com to its corresponding IP address. HTTP traffic was then generated by accessing **Google** and the **Metasploitable web server** at <http://192.168.31.242> using Firefox.

<http://192.168.31.242>
and
<http://142.251.222.206>

In Wireshark, the display filter `http` was applied to capture and analyse the HTTP packets, including request methods, URIs, and response status codes.

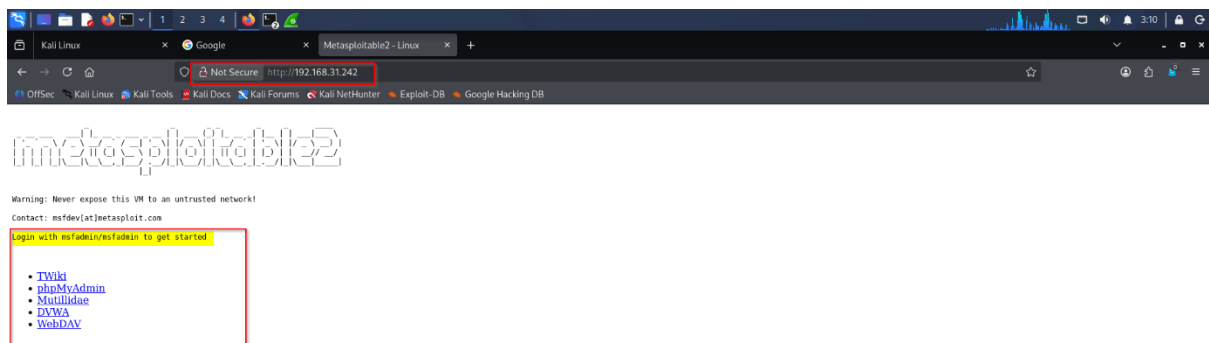


Figure 7.6(2) Metasploitable web server

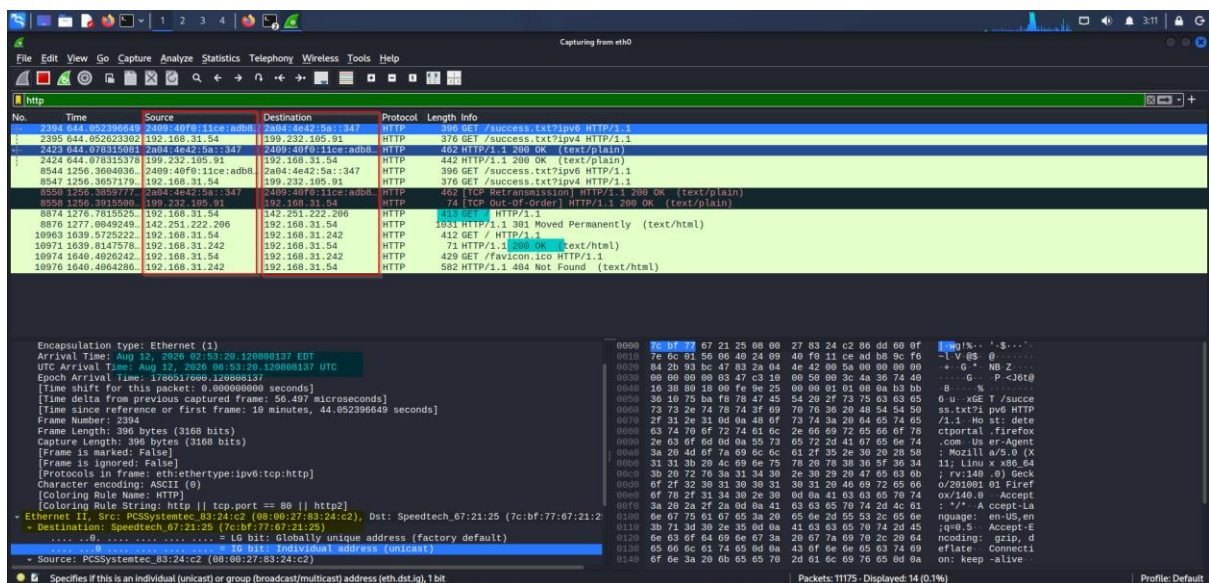


Figure 7.6(3) Live Http Packet Capture

Observation: The Wireshark capture shows HTTP communication between the client and web servers, including **GET requests** and corresponding HTTP responses. For the Metasploitable system (192.168.31.242), the capture shows **HTTP/1.1 200 OK** responses and a **404 Not Found** response for an unavailable resource. The captured packets contain details such as **source/destination IP addresses, HTTP methods, requested URI, HTTP version, and response status codes**.

The resulting DNS queries and responses were visible in the packet list. A DNS packet was selected and examined in the packet details pane, with attention to the following fields:

- Source IP address
- Destination IP address
- Transport protocol (UDP)
- Source and destination port numbers
- DNS query section
- DNS response (answer) section

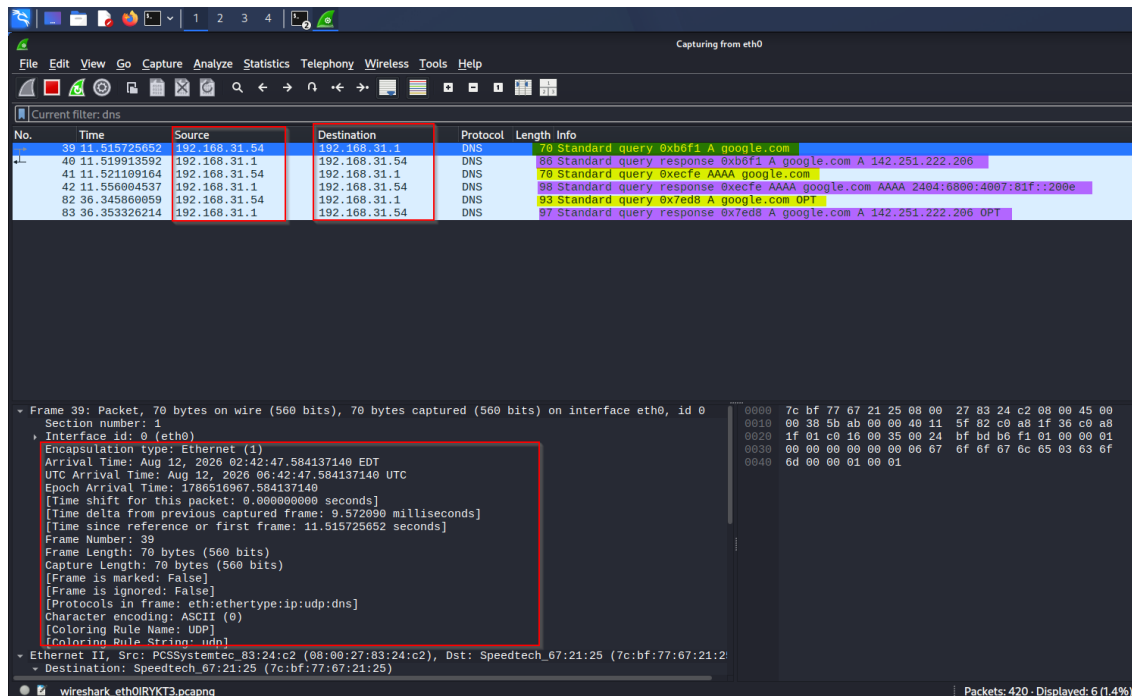


Figure 7.6(4) DNS filter applied, showing DNS packets

The Google-related capture also shows DNS resolution followed by web traffic, while the Metasploitable capture demonstrates communication with its locally hosted web application over **HTTP port 80**.

7.7 TCP Stream Reassembly

To demonstrate Wireshark's TCP stream reassembly capability, an HTTP packet was selected and, via right-click, the option Follow → TCP Stream was used. This reconstructs the full client-server conversation for that TCP connection into a single, human-readable view, rather than requiring the individual packets to be examined separately.

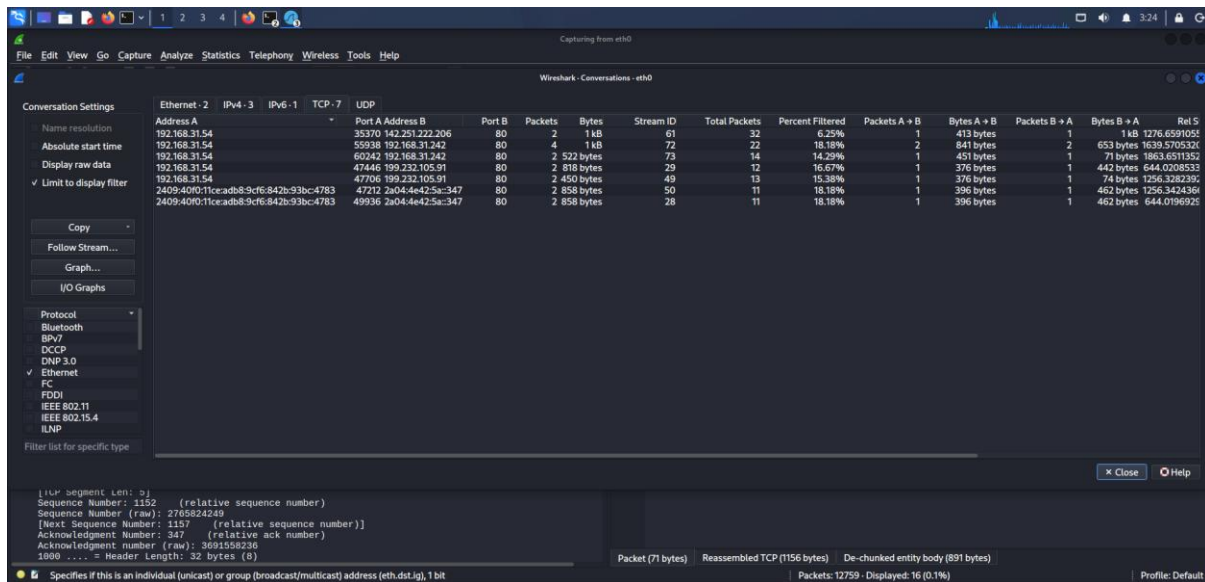


Figure 7.7 TCP Stream window showing the HTTP conversation

Observation: The Wireshark **TCP Conversations** window shows multiple HTTP connections between Kali Linux (192.168.31.54) and web servers on **port 80**. Communication with the Metasploitable system (192.168.31.242:80) is clearly visible, along with connections to external web servers. Each conversation displays the **number of packets, bytes transferred, source/destination ports, and packet direction**. The capture shows **bidirectional TCP communication**, confirming that HTTP requests and responses were exchanged between the client and web servers.

7.8 Protocol Hierarchy Statistics

Purpose: To obtain a summarised, percentage-based breakdown of the protocols present in the capture.

```
$ Statistics > Protocol Hierarchy
```

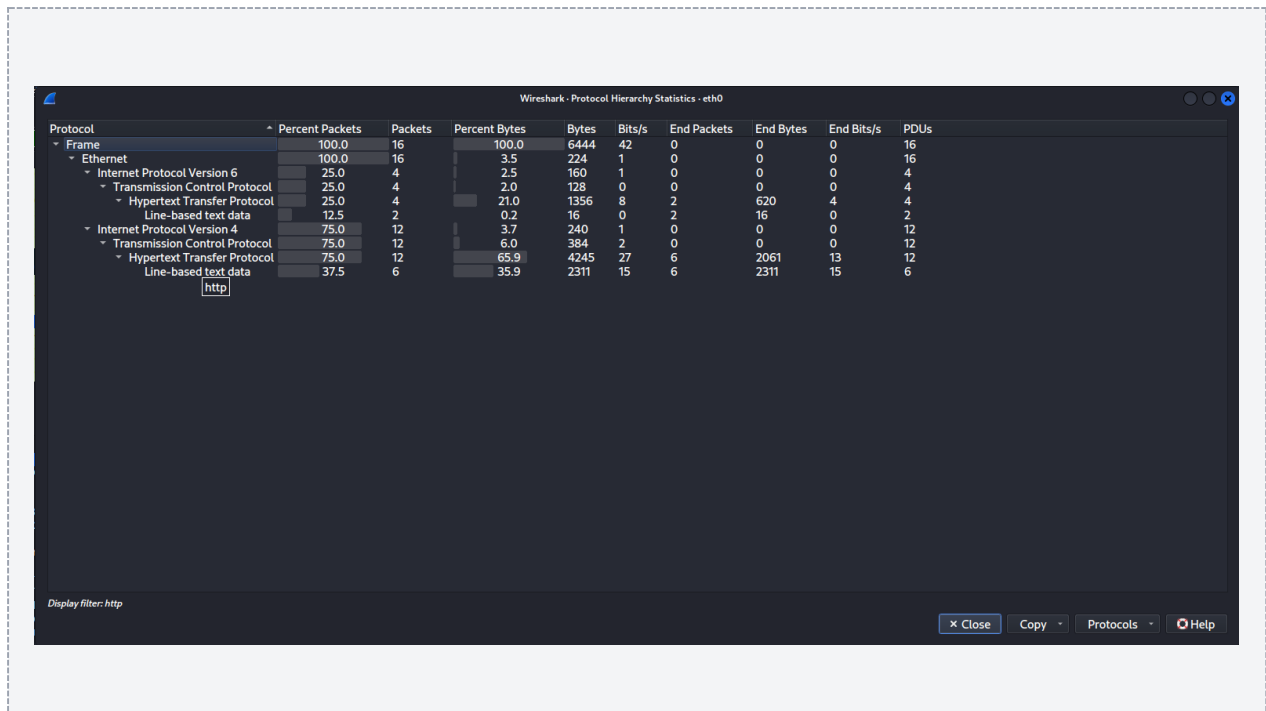


Figure 7.8 Protocol Hierarchy Statistics

Observation: The protocol hierarchy view showed TCP and UDP as the dominant transport-layer protocols, with HTTP, DNS, and ARP forming the majority of application and link-layer traffic. This view is useful for quickly understanding what a network is being used for.

7.9 Following a TCP Stream

Purpose: To reconstruct an entire client-server conversation into a single, readable view.

`$ Right-click packet > Follow > TCP Stream`

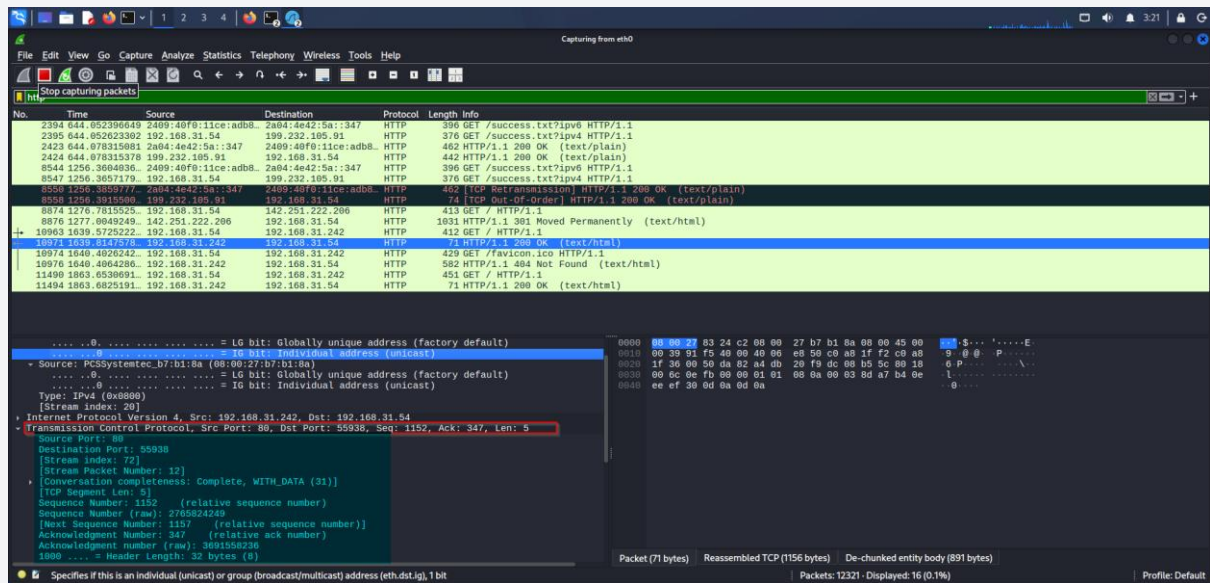


Figure 7.9 Reconstructed Session Using Follow TCP Stream

Observation: The Follow TCP Stream feature reassembled the full request-response exchange between the client and server, including headers and payload data, making it far easier to understand the application-layer conversation than reviewing individual packets.

7.10 Capturing Plaintext Credentials Over HTTP

Purpose: To demonstrate how unencrypted login forms expose credentials to anyone able to observe the traffic.

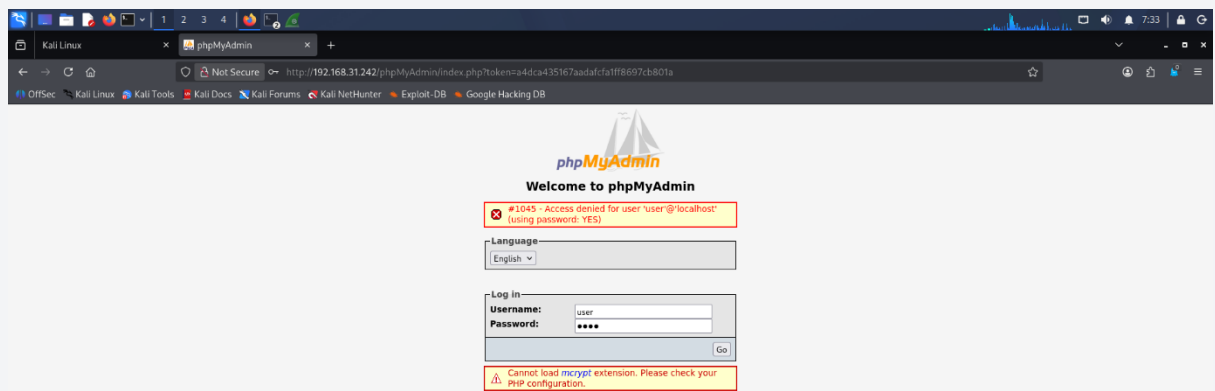
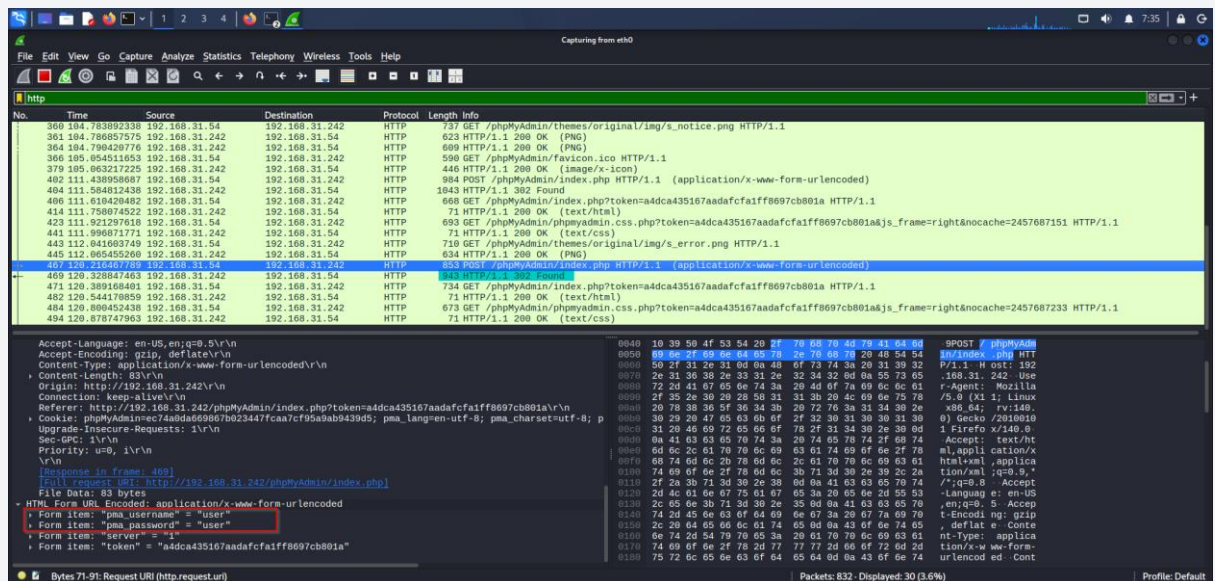


Figure 7.10(1) Plaintext Credentials Entered on website



Suggested: HTML Form URL Encoded section showing captured username and password fields

Figure 7.10(2) Plaintext Credentials Captured Over HTTP

Observation: The login form submission to the test web application (DVWA) was transmitted over plain HTTP. Inspecting the packet's HTML Form URL Encoded section revealed the submitted username and password in clear, readable credentials.

7.11 DNS Query and Response Analysis

Purpose: To examine domain name resolution activity and the information it can reveal about browsing behaviour.

\$ dns

No.	Time	Source	Destination	Protocol	Length	Info
39	11.515725652	192.168.31.54	192.168.31.1	DNS	70	Standard query 0xb6f1 A google.com
40	11.519913592	192.168.31.1	192.168.31.54	DNS	86	Standard query response 0xb6f1 A google.com A 142.251.222.206
41	11.521109164	192.168.31.54	192.168.31.1	DNS	70	Standard query response 0xb6f1 AAAA google.com
42	11.556094537	192.168.31.1	192.168.31.54	DNS	88	Standard query response 0xb6f1 AAAA google.com AAAA 2404:6800:4007:81f::200e
82	36.345860059	192.168.31.54	192.168.31.1	DNS	93	Standard query 0x7ed8 A google.com OPT
83	36.353326214	192.168.31.1	192.168.31.54	DNS	97	Standard query response 0x7ed8 A google.com A 142.251.222.206 OPT

Frame 39: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface eth0, id 0
 Section number: 1
 Interface id: 0 (eth0)
 Encapsulation type: Ethernet (1)
 Arrival Time: Aug 12, 2020 02:42:47.584137140 EDT
 UTC Arrival Time: Aug 12, 2020 06:42:47.584137140 UTC
 Epoch Arrival Time: 1786516907.584137140
 [Time shift for this packet: 0.000000000 seconds]
 [Time delta from previous captured frame: 9.572090 milliseconds]
 [Time since reference or first frame: 11.515725652 seconds]
 Frame Number: 39
 Frame Length: 70 bytes (560 bits)
 Capture Length: 70 bytes (560 bits)
 [Frame is marked: False]
 [Frame is ignored: False]
 [Protocols in frame: eth:ethertype:ip:udp:dns]
 Character encoding: ASCII (0)
 [Coloring Rule Name: UDP]
 [Coloring Rule String: udp]
 Ethernet II, Src: PCSysmteec_83:24:c2 (08:00:27:83:24:c2), Dst: Speedtech_67:21:25 (7c:bf:77:67:21:25)
 Destination: Speedtech_67:21:25 (7c:bf:77:67:21:25)

Figure 7.11 DNS Query and Response Analysis

Observation: The DNS filter revealed each domain name resolved by the test host, along with the corresponding IP addresses and TTL values returned. Because DNS traffic is unencrypted by default, this confirmed that an observer on the network could reconstruct a fairly detailed picture of a user's browsing activity.

7.12 Summary of Findings

The captures reveal that the target test environment transmits several types of sensitive and diagnostic information in the clear. The most significant finding is that login credentials submitted over HTTP were fully readable within the packet capture, demonstrating a direct and practical risk of credential theft on unencrypted or improperly secured web applications. DNS queries further exposed a detailed record of domain lookups, and ARP traffic showed the kind

of address-resolution activity that, if manipulated, could enable interception of local traffic.

Overall Risk Rating: High. Remediation in a production environment would require enforcing HTTPS/TLS for all authentication traffic, disabling legacy plaintext protocols, and implementing monitoring for anomalous ARP and DNS behaviour.

Chapter 8: Risk Analysis

8.1 Overview

The packet captures conducted during this project revealed several categories of traffic on the target test network. While the purpose of the captures was to assess the security posture of an authorized test environment, the results demonstrate how unencrypted or poorly monitored traffic can present real security risks if left unaddressed.

8.2 Risk Identified

Risk Identified	Potential Impact	Severity	Recommended Mitigation
Plaintext HTTP Credentials	Credential theft via traffic interception	Critical	Enforce HTTPS/TLS for all login and authentication traffic.
Unencrypted FTP/Telnet Traffic	Exposure of credentials and session data	Critical	Replace with SFTP/SSH; disable legacy plaintext protocols.
DNS Query Exposure	Disclosure of browsing behaviour and internal hostnames	Medium	Use DNS over HTTPS/TLS (DoH/DoT) where feasible.
ARP Spoofing Susceptibility	Man-in-the-middle traffic interception	High	Enable Dynamic ARP Inspection or use static ARP entries on critical hosts.
Unrestricted Packet Capture Access	Unauthorized users could passively monitor sensitive traffic	Medium	Restrict capture privileges and physical/network access to authorized personnel.

8.3 Risk Assessment

The practical captures demonstrated that even a small test network carries a meaningful amount of sensitive information in the clear. Regular packet-level auditing helps identify such exposures before they can be exploited, and

complements log-based and signature-based monitoring by revealing exactly what is transmitted at the byte level.

8.4 Security Recommendations

- Enforce HTTPS/TLS for all web-based authentication and sensitive data transfer.
- Disable insecure legacy protocols such as Telnet and unencrypted FTP.
- Perform regular, authorized packet captures to audit traffic on critical network segments.
- Enable Dynamic ARP Inspection or static ARP entries to mitigate spoofing risk.
- Restrict who can run packet-capture tools on production networks.
- Monitor DNS traffic for unusual query patterns or connections to suspicious domains.
- Maintain documentation of capture findings and remediation actions taken.

Chapter 9: Conclusion

This project provided a comprehensive understanding of Wireshark and its role in packet capture, protocol analysis, and network security investigation. Practical implementation demonstrated live traffic capture, the use of capture and display filters, protocol hierarchy statistics, TCP stream reconstruction, and the identification of plaintext credentials, DNS activity, and ARP-based risks.

The captures highlighted the importance of encrypting sensitive traffic, retiring legacy plaintext protocols, and maintaining visibility into address-resolution behaviour on the local network. Appropriate remediation measures such as enforcing TLS, disabling insecure protocols, and monitoring for ARP anomalies can significantly improve an organization's security posture.

Overall, Wireshark is an essential tool for cybersecurity professionals, network administrators, and digital forensics investigators. When used responsibly and with proper authorization, it supports proactive security assessments, effective troubleshooting, and thorough incident investigation. This project has enhanced our practical knowledge of network traffic analysis and provides a strong foundation for future work in network security and digital forensics.